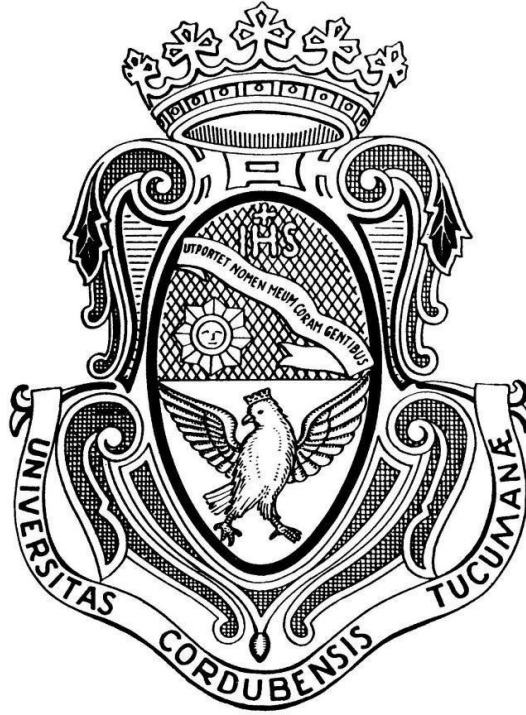


Universidad Nacional de Córdoba

Facultad de Ciencias Exactas, Físicas y Naturales



Sistemas de Computación

Trabajo Práctico N° 3.a - Arquitectura y Fundamentos del Firmware UEFI

Alumno: Gastón Yomaha

Docentes: Javier Jorge
Miguel Solinas

22 de mayo de 2026

Índice

1	Exploración del Entorno UEFI y la Shell	2
1.1	Arranque en el Entorno Virtual	2
1.2	Exploración de Dispositivos (Handles y Protocolos)	2
1.3	Análisis de Variables Globales (NVRAM)	4
1.4	Footprinting de Memoria y Hardware	5
2	Desarrollo, Compilación y Análisis de Seguridad	8
2.1	Desarrollo de la Aplicación	8
2.2	Compilación a Formato PE/COFF	8
2.3	Análisis de Metadatos y Decompilación	9
2.4	Ejecutar aplicacion.efi en QEMU	12
3	Ejecución en Hardware Físico (Bare Metal)	14
3.1	Preparación del Medio de Arranque (USB)	14
3.2	Configuración del BIOS/UEFI (ASUS X515EA)	16
3.3	Ejecución en Bare Metal	17
4	Depuración con GDB	19
4.1	Depuración con GDB más QEMU	19
4.2	Conectar Ghidra a GDB (Análisis Híbrido)	22
5	Conclusiones	23
6	Link al Repositorio	24

1. Exploración del Entorno UEFI y la Shell

Objetivo: Explorar cómo UEFI abstrae el hardware y gestiona la configuración antes de la carga del sistema operativo.

1.1. Arranque en el Entorno Virtual

A diferencia del BIOS Legacy que simplemente leía el primer sector de un disco (MBR), UEFI es un entorno completo con su propio gestor de memoria, red y consola.

Se configura el emulador QEMU mediante el siguiente comando:

```
qemu-system-x86_64 -m 512 -bios /usr/share/ovmf/OVMF.fd -net none
```

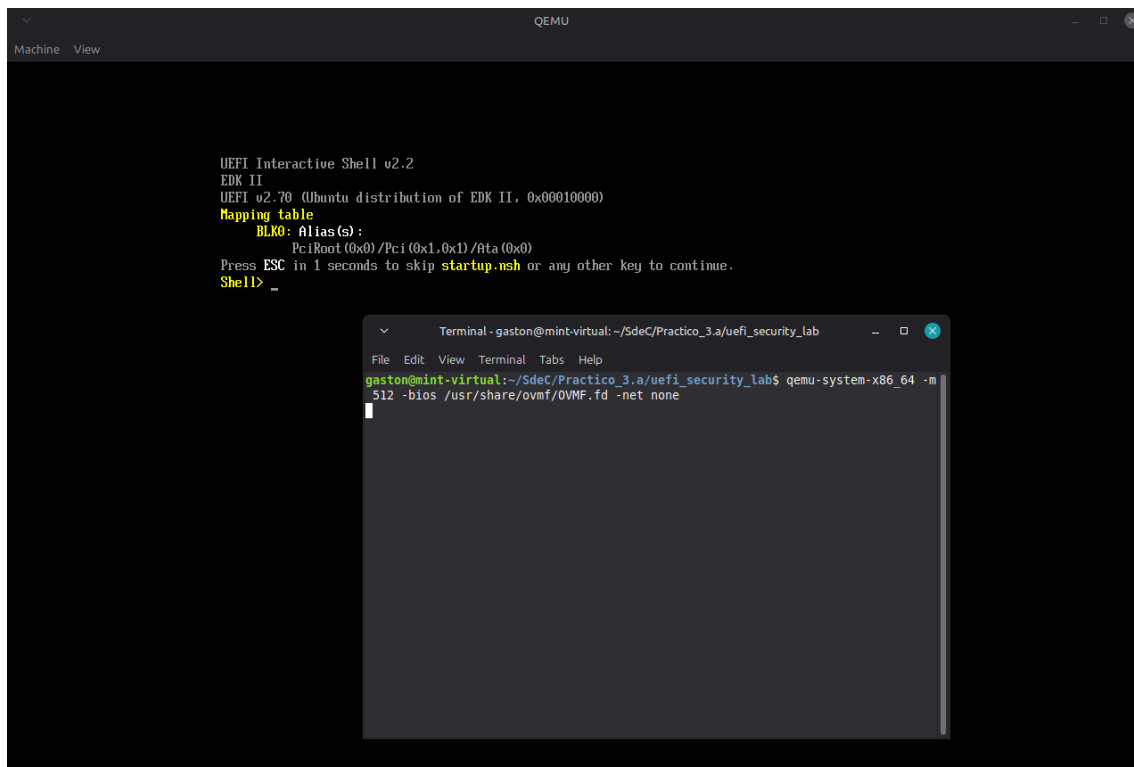


Figura 1: Configuración de QEMU.

1.2. Exploración de Dispositivos (Handles y Protocolos)

UEFI no usa letras de unidad fijas (como C:). Mantiene una base de datos de “Handles” (identificadores) que agrupan “Protocolos” (interfaces de software como SIMPLE_FILE_SYSTEM).

Comandos de la UEFI Shell:

```
Shell> map
```

Este comando muestra la tabla de mapeo de dispositivos, creando alias cortos para facilitar la interacción con un usuario.

```
Shell> dh -b
```

Este comando significa Device Handles, y el parámetro -b se utiliza para detener el texto cuando se llene la terminal. El comando se utiliza para mostrar una lista de “Handles” y los “Protocolos” que están cargados en la memoria UEFI.

```

QEMU

UEFI Interactive Shell v2.2
EDK II
UEFI v2.70 (Ubuntu distribution of EDK II, 0x00010000)
Mapping table
  BLK0: Alias(s) :
        PciRoot (0x0) /Pci (0x1,0x1) /Ata (0x0)
Press ESC in 1 seconds to skip startup.nsh or any other key to continue.
Shell> map
Mapping table
  BLK0: Alias(s) :
        PciRoot (0x0) /Pci (0x1,0x1) /Ata (0x0)
Shell> _

```

Figura 2: Comando map.

```

QEMU

Handle dump
01: LoadedImage (DxeCore)
02: Decompress
03: FirmwareVolume2 DevicePath(..F8EB-4F34-AAEA-3EE4AF6516A1)) FirmwareVolumeBlock
04: EE4E5898-3914-4259-9D6E-DC7BD79403CF
05: ImageDevicePath(..AD6B-4F3A-B60B-F59899003443)) LoadedImage (DevicePathDxe)
06: DevicePathFromText DevicePathToText DevicePathUtilities
07: ImageDevicePath(..87AB-47F9-A3FE-D50B76D89541)) LoadedImage (PcdDxe)
08: GetPcdInfo GetPcdInfoProtocol Pcd Pcd
09: BB00A5CA-08CE-462F-A537-43C74A825CA4 ImageDevicePath(..7BCE-40AF-B5BB-47E3611EB5C2)) LoadedImage
(TdxDxe)
0A: ImageDevicePath(..A7EB-4730-8C8E-CC466A9ECC3C)) LoadedImage (ReportStatusCodeRouterRuntimeDxe)
0B: SmartCardReader RscHandler
0C: ImageDevicePath(..43B7-4784-95B1-F4226CB40CEE)) LoadedImage (RuntimeDxe)
0D: RuntimeArch
0E: ImageDevicePath(..7FD6-4665-8646-88E33EF71DFC)) LoadedImage (SecurityStubDxe)
0F: SecurityArch Security2Arch
10: DeferredImageLoad
11: DebugSupport 96F46153-97A7-4793-ACC1-FA19BF78EA97 EBCInterpreter ImageDevicePath(..73D0-11D4-B06
B-00AA00BD6DE7)) LoadedImage (EbcDxe)
12: AAEEACCFD-F27B-4C17-B610-75CA1F2DFB52
13: ImageDevicePath(..C1BC-49F8-875F-54A5D542443F)) LoadedImage (CpuIo2Dxe)
14: CpuIo2
15: ImageDevicePath(..9517-440E-9FDE-3BE44CEE2136)) LoadedImage (CpuDxe)
16: CpuArch
17: MpService
18: IncompatiblePciDeviceSupport ImageDevicePath(..A776-4EE1-B643-1FEFF2B615BB)) LoadedImage (Incompa
tiblePciDeviceSupportDxe)
19: PciHotPlugInit ImageDevicePath(..A9BE-426D-A6CC-B22FE51D9224)) LoadedImage (PciHotPlugInitDxe)
1A: ImageDevicePath(..FF36-4E10-93CF-A82159E777C5)) LoadedImage (ResetSystemRuntimeDxe)
Press ENTER to continue or 'Q' break:_

```

Figura 3: Comando dh -b.

Pregunta de Razonamiento 1: Al ejecutar el comando map y dh, vemos protocolos e identificadores en lugar de puertos de hardware fijos. ¿Cuál es la ventaja de seguridad de este modelo frente al antiguo BIOS?

Respuesta 1: El antiguo BIOS funcionaba en modo real de 16 bits, de forma que cualquier programa que se ejecutara durante el arranque tenía acceso a toda la memoria y los puertos de hardware.

Al obligar a que todo pase a través de “Protocolos” (que son básicamente APIs o funciones programadas), UEFI puede aumentar la seguridad. Como no se puede acceder al hardware directamente, el software malicioso está obligado a usar los protocolos de la UEFI. Esto permite implementar medidas como Secure Boot, verificando que el software que intenta usar el protocolo del disco tenga una firma criptográfica válida.

1.3. Análisis de Variables Globales (NVRAM)

La fase BDS (Boot Device Selection) decide qué cargar basándose en variables no volátiles.

Comandos de la UEFI Shell:

```
Shell> dmpstore
```

Este comando significa Dump Store, y consiste en leer y mostrar en pantalla todas las variables almacenadas en la NVRAM (memoria no volátil, que no se borra al apagar el equipo). En este lugar se guardan las configuraciones críticas del sistema, como por ejemplo claves del Secure Boot.

```

QEMU

000001E0: 01 01 06 00 00 01 02 01-0C 00 D0 41 01 05 00 00 *.....Å....*
000001F0: 00 00 03 0E 13 00 00 00-00 00 00 C2 01 00 00 00 *.....*
00000200: 00 00 08 01 01 03 0A 14-00 6B A5 FC FB 36 BB 78 *.....k...6.x*
00000210: 4B AA AB BE 1B 97 EC 7C-CB 7F 01 04 00 02 01 0C *K.....l.....*
00000220: 00 D0 41 03 0A 00 00 00-00 01 01 06 00 00 01 02 *..Å.....*
00000230: 01 0C 00 D0 41 01 05 00-00 00 00 03 0E 13 00 00 *....Å.....*
00000240: 00 00 00 00 C2 01 00 00-00 00 00 08 01 01 03 0A *.....*
00000250: 14 00 DD DD 46 8E 49 3D-9D 4A B8 75 3C 08 6F 6A *....F.I=.J.u<.oj*
00000260: A2 BD 7F 01 04 00 02 01-0C 00 D0 41 03 0A 00 00 *.....Å....*
00000270: 00 00 01 01 06 00 00 01-02 01 0C 00 D0 41 01 05 *.....Å..*
00000280: 00 00 00 03 0E 13 00-00 00 00 00 00 C2 01 00 *.....*
00000290: 00 00 00 08 01 01 03-0A 14 00 E0 D6 7D FC 3C *.....}<.*
000002A0: 81 4D 43 B4 DA 3B D6 49-E9 E1 5A 7F FF 04 00 *..MC...;I..Z....*
Variable RT+BS 'EFIGlobalVariable:PlatformRecovery0000' DataSize = 0x6C
00000000: 01 00 00 00 34 00 44 00-65 00 66 00 61 00 75 00 *...4.D.e.f.a.u.*
00000010: 6C 00 74 00 20 00 50 00-6C 00 61 00 74 00 66 00 *l.t. .P.l.a.t.f.*
00000020: 6F 00 72 00 6D 00 52 00-65 00 63 00 6F 00 76 00 *o.r.m.R.e.c.o.v.*
00000030: 65 00 72 00 79 00 00 00-04 04 30 00 5C 00 45 00 *e.r.y....0.\.E.*
00000040: 46 00 49 00 5C 00 42 00-4F 00 4F 00 54 00 5C 00 *F.I.\.B.O.O.T.\.*
00000050: 42 00 4F 00 4F 00 54 00-58 00 36 00 34 00 2E 00 *B.O.O.T.X.6.4...*
00000060: 45 00 46 00 49 00 00 00-7F FF 04 00 *E.F.I.....*
Variable RT+BS 'EFIGlobalVariable:PlatformLangCodes' DataSize = 0x12
00000000: 65 6E 3B 66 72 3B 65 6E-2D 55 53 3B 66 72 2D 46 *en;fr;en-US;fr-F*
00000010: 52 00 *R.*
Variable RT+BS 'EFIGlobalVariable:LangCodes' DataSize = 0x0D
00000000: 65 6E 67 66 72 61 65 6E-67 66 72 61 00 *engfraengfra.*
Variable RT+BS 'EFIGlobalVariable:BootOptionSupport' DataSize = 0x04
00000000: 13 03 00 00 *...*
Variable RT+BS 'EFIGlobalVariable:OsIndicationsSupported' DataSize = 0x08
00000000: 41 00 00 00 00 00 00 00- *Å.....*
Shell> _
    
```

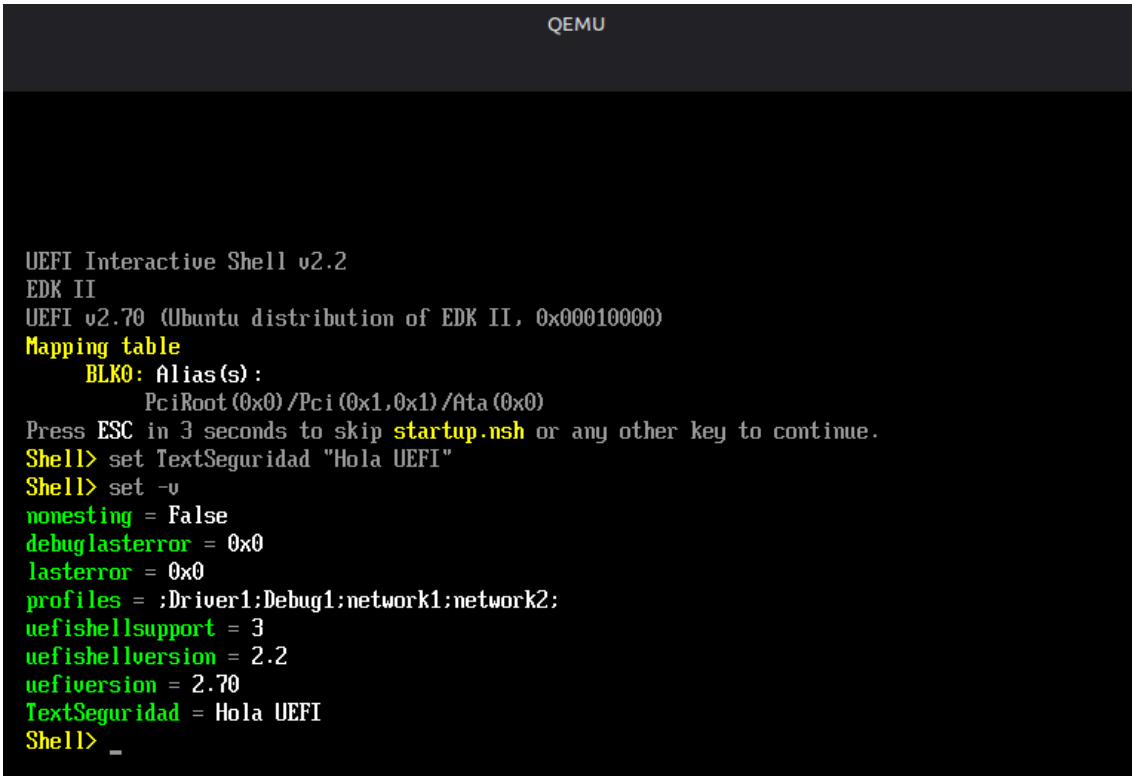
Figura 4: Comando dmpstore.

```
Shell> set TextSeguridad '‘Hola UEFI’'
```

Este comando se utiliza para crear o modificar variables de entorno dentro de la sesión actual de la Shell. Entonces, se está creando una variable llamada TextSeguridad y se le asigna el texto “Hola UEFI”.

```
Shell> set -v
```

Este comando muestra en pantalla la lista de variables de entorno volátiles actuales (es decir, las que están presentes cuando la Shell está abierta). En este caso se permite verificar que la variable TextSeguridad se creó correctamente.



```
QEMU

UEFI Interactive Shell v2.2
EDK II
UEFI v2.70 (Ubuntu distribution of EDK II, 0x00010000)
Mapping table
  BLK0: Alias(s) :
      PciRoot(0x0)/Pci(0x1,0x1)/Ata(0x0)
Press ESC in 3 seconds to skip startup.nsh or any other key to continue.
Shell> set TextSeguridad "Hola UEFI"
Shell> set -v
nonesting = False
debuglasterror = 0x0
lasterror = 0x0
profiles = ;Driver1;Debug1;network1;network2;
uefishellsupport = 3
uefishellversion = 2.2
uefiversion = 2.70
TextSeguridad = Hola UEFI
Shell> _
```

Figura 5: Comandos set.

Pregunta de Razonamiento 2: Observando las variables Boot#### y BootOrder, ¿cómo determina el Boot Manager la secuencia de arranque?

Respuesta 2: Lo primero que hace el Boot Manager es leer la variable BootOrder, la cual contiene una lista ordenada de números hexadecimales de 4 dígitos. Luego, el Boot Manager se encarga de buscar la variable Boot#### (uno de los números de la lista en BootOrder), la cual contiene la ruta hacia el archivo ejecutable del cargador del arranque. Si el intento de arranque falla, se elige otra variable dentro de la lista de BootOrder y se repite el proceso.

1.4. Footprinting de Memoria y Hardware

Comandos de la UEFI Shell:

```
Shell> memmap -b
```

Este comando muestra el mapa de memoria del sistema. Le pide a la UEFI que imprima una lista detallada de cómo está dividida la RAM en ese momento, indicando qué bloques están libres, reservados para hardware o siendo usados por el firmware.

```

QEMU

Type      Start      End      # Pages  Attributes
BS_Code   0000000000000000-000000000000FFFF 0000000000000001 000000000000000F
Available 0000000000001000-0000000000009FFF 000000000000009F 000000000000000F
Available 00000000000010000-0000000000007FFFF 00000000000000700 000000000000000F
ACPI_NVS   00000000000080000-00000000000087FFF 0000000000000008 000000000000000F
Available 00000000000080000-00000000000080AFF 0000000000000003 000000000000000F
ACPI_NVS   00000000000080000-00000000000080BFFF 0000000000000001 000000000000000F
Available 00000000000080C000-00000000000080FFF 0000000000000004 000000000000000F
ACPI_NVS   000000000000810000-0000000000008FFFF 00000000000000F0 000000000000000F
BS_Data    00000000000090000-000000000000177FFF 00000000000000E80 000000000000000F
Available 0000000000001780000-0000000000001BB74FFF 0000000000001A3F5 000000000000000F
BS_Data    0000000000001BB75000-0000000000001BB94FFF 00000000000000020 000000000000000F
Available 0000000000001BB95000-0000000000001E3CBFFF 00000000000002037 000000000000000F
LoaderCode 0000000000001E3CC000-0000000000001E4AFFF 00000000000000D9 000000000000000F
Available 0000000000001E4A5000-0000000000001E50CFFF 00000000000000068 000000000000000F
BS_Data    0000000000001E50D000-0000000000001E52AFFF 0000000000000001E 000000000000000F
Available 0000000000001E52B000-0000000000001E53BFFF 00000000000000009 000000000000000F
BS_Data    0000000000001E53C000-0000000000001E56CFFF 00000000000000139 000000000000000F
BS_Code    0000000000001E56D000-0000000000001E74DFFF 000000000000000E1 000000000000000F
BS_Data    0000000000001E74E000-0000000000001E7B9FFF 0000000000000006C 000000000000000F
BS_Code    0000000000001E7BA000-0000000000001E7C2FFF 00000000000000009 000000000000000F
BS_Data    0000000000001E7C3000-0000000000001E7C7FFF 00000000000000005 000000000000000F
BS_Code    0000000000001E7C8000-0000000000001E806FFF 0000000000000003F 000000000000000F
BS_Data    0000000000001E807000-0000000000001E809FFF 00000000000000003 000000000000000F
BS_Code    0000000000001E80A000-0000000000001E80CFFF 00000000000000003 000000000000000F
BS_Data    0000000000001E80D000-0000000000001E813FFF 00000000000000007 000000000000000F
BS_Code    0000000000001E814000-0000000000001E825FFF 00000000000000012 000000000000000F
BS_Data    0000000000001E826000-0000000000001E826FFF 00000000000000001 000000000000000F
BS_Code    0000000000001E827000-0000000000001E829FFF 00000000000000003 000000000000000F
BS_Data    0000000000001E82A000-0000000000001E82EFFF 00000000000000005 000000000000000F
Press ENTER to continue or 'Q' break: _

```

Figura 6: Comando memmap -b parte 1.

```

QEMU

ACPI_Recl   000000001F76D000-000000001F77EFFF 0000000000000012 000000000000000F
ACPI_NVS    0000000001F77F000-0000000001F7FEFFF 00000000000000080 000000000000000F
BS_Data     0000000001F7FF000-0000000001FDFFFF 000000000000000601 000000000000000F
Available   0000000001FE00000-0000000001FE76FFF 00000000000000077 000000000000000F
BS_Data     0000000001FE77000-0000000001FE96FFF 00000000000000020 000000000000000F
BS_Code     0000000001FE97000-0000000001FEC9FFF 00000000000000033 000000000000000F
Press ENTER to continue or 'Q' break:
BS_Data     0000000001FECB000-0000000001FEDAFF 0000000000000011 000000000000000F
BS_Code     0000000001FEDB000-0000000001FEF3FFF 00000000000000019 000000000000000F
RT_Data     0000000001FEF4000-0000000001FFF7FFF 00000000000000084 000000000000000F
ACPI_NVS    0000000001FFF8000-0000000001FFFFFF 00000000000000088 000000000000000F
Reserved    000000FD00000000-000000FFFFFF 0000000000300000 0000000000000000

Reserved :      3,145,856 Pages (12,885,426,176 Bytes)
LoaderCode:      217 Pages (888,832 Bytes)
LoaderData:       0 Pages (0 Bytes)
BS_Code :        850 Pages (3,481,600 Bytes)
BS_Data :        8,419 Pages (34,484,224 Bytes)
RT_Code :        256 Pages (1,048,576 Bytes)
RT_Data :        581 Pages (2,379,776 Bytes)
ACPI_Recl :      18 Pages (73,728 Bytes)
ACPI_NVS :       513 Pages (2,101,240 Bytes)
MMIO :           0 Pages (0 Bytes)
MMIO_Port :      0 Pages (0 Bytes)
PalCode :        0 Pages (0 Bytes)
Unaccepted:      0 Pages (0 Bytes)
Available :     119,994 Pages (491,495,424 Bytes)
Persistent:       0 Pages (0 Bytes)

Total Memory:      511 MB (535,953,408 Bytes)
Shell> _

```

Figura 7: Comando memmap -b parte 2.

```
Shell> pci -b
```

Este comando muestra información sobre todos los dispositivos conectados a los buses PCI y PCIe (Peripheral Component Interconnect), como tarjetas gráficas, controladores de red, de discos, etc.

```
Shell> pci -b
  Seg  Bus  Dev  Func
  ---  ---  ---  ---
    00   00   00   00 ==> Bridge Device - Host/PCI bridge
      Vendor 8086 Device 1237 Prog Interface 0
    00   00   01   00 ==> Bridge Device - PCI/ISA bridge
      Vendor 8086 Device 7000 Prog Interface 0
    00   00   01   01 ==> Mass Storage Controller - IDE controller
      Vendor 8086 Device 7010 Prog Interface 80
    00   00   01   03 ==> Bridge Device - Other bridge type
      Vendor 8086 Device 7113 Prog Interface 0
    00   00   02   00 ==> Display Controller - VGA/8514 controller
      Vendor 1234 Device 1111 Prog Interface 0
Shell> _
```

Figura 8: Comando pci -b.

```
Shell> drivers -b
```

Este comando imprime una lista de todos los drivers UEFI que están cargados actualmente en la memoria y funcionando.

```
QEMU

          T  D
          Y  C I
          P  F A
          U
=====
#D #C DRIVER NAME                                IMAGE NAME
=====
59 0000000A B - - 1 5 PCI Bus Driver              PciBusDxe
5B 00000010 ? - - - - Virtio PCI Driver          VirtioPciDeviceDxe
5C 00000010 ? - - - - Virtio 1.0 PCI Driver      Virtio10
5D 00000010 ? - - - - Virtio Block Driver        VirtioBlkDxe
5E 00000010 ? - - - - Virtio SCSI Host Driver    VirtioScsiDxe
5F 00000010 ? - - - - Virtio Random Number Generator Driv VirtioRngDxe
60 00000010 ? - - - - Virtio Serial Driver      VirtioSerialDxe
61 0000000A D - - 2 - Platform Console Management Driver ConPlatformDxe
62 0000000A D - - 2 - Platform Console Management Driver ConPlatformDxe
63 0000000A B - - 2 2 Console Splitter Driver     ConSplitterDxe
64 0000000A ? - - - - Console Splitter Driver     ConSplitterDxe
65 0000000A ? - - - - Console Splitter Driver     ConSplitterDxe
66 0000000A B - - 2 2 Console Splitter Driver     ConSplitterDxe
67 0000000A B - - 1 1 Console Splitter Driver     ConSplitterDxe
6B 0000000A D - - 1 - Graphics Console Driver     GraphicsConsoleDxe
6C 0000000A B - - 1 1 Serial Terminal Driver      TerminalDxe
6D 0000000A D - - 1 - Generic Disk I/O Driver      DiskIoDxe
6E 0000000B ? - - - - Partition Driver (MBR/GPT/El Torito) PartitionDxe
71 0000000A B - - 1 1 SCSI Bus Driver             ScsiBus
72 0000000A D - - 1 - Scsi Disk Driver             ScsiDisk
73 0000000A D - - 1 - Sata Controller Init Driver  SataController
74 00000010 D - - 1 - AtaAtapiPassThru Driver     AtaAtapiPassThruDxe
75 00000010 D - - 1 - ATA Bus Driver              AtaBusDxe
76 00000010 ? - - - - NUM Express Driver          NumExpressDxe
77 00000010 B - - 1 3 OUMF Sio Bus Driver          SioBusDxe
Press ENTER to continue or 'Q' break: _
```

Figura 9: Comando drivers -b.

Pregunta de Razonamiento 3: En el mapa de memoria (memmap), existen regiones marcadas como RuntimeServicesCode. ¿Por qué estas áreas son un objetivo principal para los desarrolladores de malware (Bootkits)?

Respuesta 3: Mientras que la memoria de Boot Services se borra cuando se termina de encender el equipo, la memoria de Runtime Services contiene funciones críticas que el SO necesita consultar durante el funcionamiento. Entonces, si se logra inyectar un código malicioso en esta región de memoria, el virus sobrevive el proceso de arranque, y también tiene los privilegios más altos posibles.

2. Desarrollo, Compilación y Análisis de Seguridad

Objetivo: Crear una aplicación nativa UEFI en C, entender el formato PE/COFF y analizar cómo un descompilador interpreta opcodes a nivel de firmware.

2.1. Desarrollo de la Aplicación

Código C:

```
#include <efi.h>
#include <efilib.h>

EFI_STATUS EFIAPI efi_main(EFI_HANDLE ImageHandle, EFI_SYSTEM_TABLE* SystemTable){
    volatile unsigned char code[] = {
        0xCC
    };
    uefi_call_wrapper(SystemTable->ConOut->ClearScreen, 1, SystemTable->ConOut);
    uefi_call_wrapper(SystemTable->ConOut->OutputScreen, 2, SystemTable->ConOut,
        L "Iniciando analisis de seguridad...\r\n");
    if(code[0] == 0xCC){
        uefi_call_wrapper(SystemTable->ConOut->OutputScreen, 2, SystemTable->ConOut,
            L "Breakpoint estatico alcanzado.\r\n");
    }
    uefi_call_wrapper(SystemTable->BootServices->Stall, 1, 5000000);
    return EFI_SUCCESS;
}
```

Pregunta de Razonamiento 4: ¿Por qué utilizamos SystemTable->ConOut->OutputString en lugar de la función printf de C?

Respuesta 4: La función printf pertenece a la biblioteca de C <stdio.h>, y se encarga de comunicarse con el SO usando system calls para lograr que se dibujen letras en la pantalla. En cambio, cuando se ejecuta el código UEFI, el SO está apagado, por lo que no hay bibliotecas cargadas en memoria, ni hay un kernel al cual pedirle que imprima cosas. Es por ello que es necesario utilizar la SystemTable, que contiene funciones primitivas como OutputString.

2.2. Compilación a Formato PE/COFF

UEFI utiliza el formato OE/COFF (típico de Windows), incluso si compilamos desde Linux.

Comandos:

```
# 1. Compilar a código objeto
gcc -I/usr/include/efi -I/usr/include/efi/x86_64 -I/usr/include/efi/protocol -fpic
-ffreestanding -fno-stack-protector -fno-strict-aliasing -fshort-wchar -mno-red-zone
-maccumulate-outgoing-args -Wall -c -o aplicacion.o aplicacion.c

# 2. Linkear (generar .so intermedio)
ld -shared -Bsymbolic -L/usr/lib -L/usr/lib/efi -T /usr/lib/elf_x86_64_efi.lds /usr/lib/
crt0-efi-x86_64.o aplicacion.o -o aplicacion.so -lefi -lgmefi

# 3. Convertir a ejecutable EFI (PE/COFF)
objcopy -j .text -j .sdata -j .data -j .dynamic -j .dynsym -j .rel -j .rela -j .rel.* -j
.rela.* -j .reloc --target=efi-app-x86_64 aplicacion.so aplicacion.efi
```

```

Terminal - gaston@mint-virtual: ~/SdeC/Practico_3.a/uefi_security_lab
File Edit View Terminal Tabs Help
gaston@mint-virtual:~/SdeC/Practico_3.a/uefi_security_lab$ # 1. Compilar a código objeto
gcc -I/usr/include/efi -I/usr/include/efi/x86_64 -I/usr/include/efi/protocol -fpic -ffreestanding -ggdb -c *.c
gaston@mint-virtual:~/SdeC/Practico_3.a/uefi_security_lab$ # 2. Linkear (generar .so intermedio)
ld -shared -Bsymbolic -L/usr/lib -L/usr/lib/efi -T /usr/lib/elf_x86_64_efi.lds /usr/lib/crt0-efi-x86_64.o aplicacion.o -o aplicacion.so -lefi -lgnuv1
gaston@mint-virtual:~/SdeC/Practico_3.a/uefi_security_lab$ # 3. Convertir a ejecutable EFI (PE/COFF)
objcopy -j .text -j .sdata -j .data -j .dynamic -j .dynsym -j .rel -j .rela -j .rel.* -j .rela.* -j .r
eoloc --target=efi-app-x86_64 aplicacion.so aplicacion.efi
gaston@mint-virtual:~/SdeC/Practico_3.a/uefi_security_lab$
  
```

Figura 10: Comandos de Compilación, Linkeo y Conversión a Ejecutable.

2.3. Análisis de Metadatos y Decompilación

Comandos:

```
file aplicacion.efi
```

```

Terminal - gaston@mint-virtual: ~/SdeC/Practico_3.a/uefi_security_lab
File Edit View Terminal Tabs Help
gaston@mint-virtual:~/SdeC/Practico_3.a/uefi_security_lab$ file aplicacion.efi
aplicacion.efi: PE32+ executable (EFI application) x86-64 (stripped to external PDB), for MS Windows, 5 sections
gaston@mint-virtual:~/SdeC/Practico_3.a/uefi_security_lab$
  
```

Figura 11: Comando file.

A continuación se detalla el procedimiento para utilizar el software GHIDRA:
Primero se crea el proyecto, indicando su ruta:

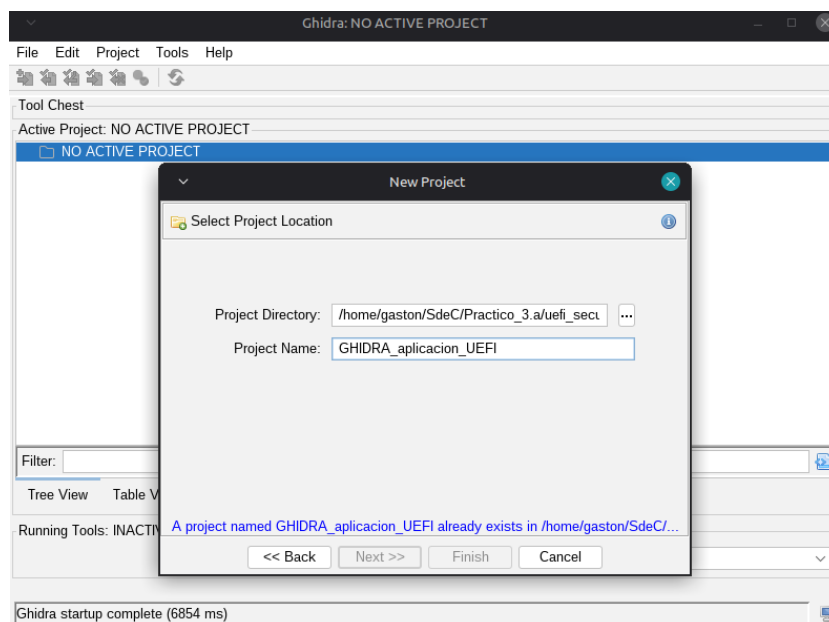


Figura 12: Proyecto en GHIDRA.

Luego se importa el archivo aplicacion.efi:

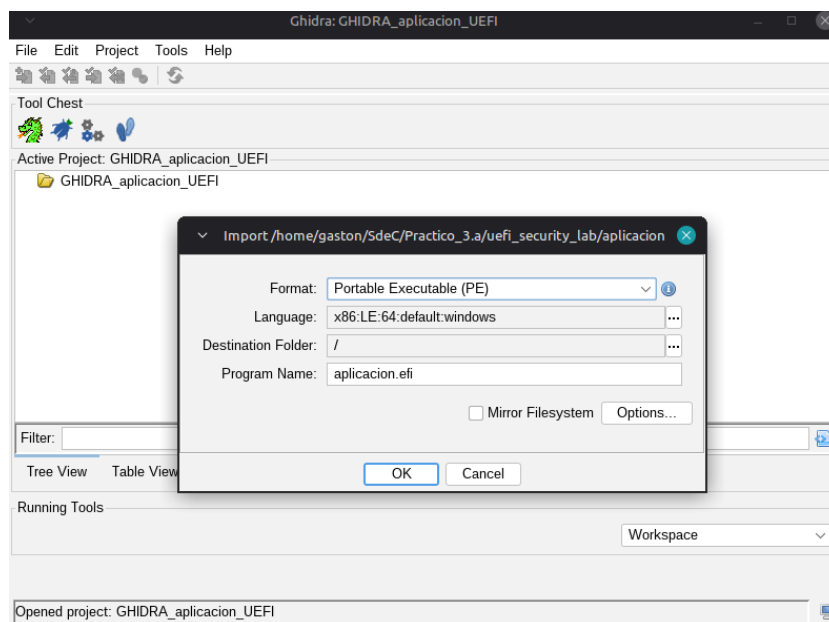


Figura 13: Importar archivo .efi en GHIDRA.

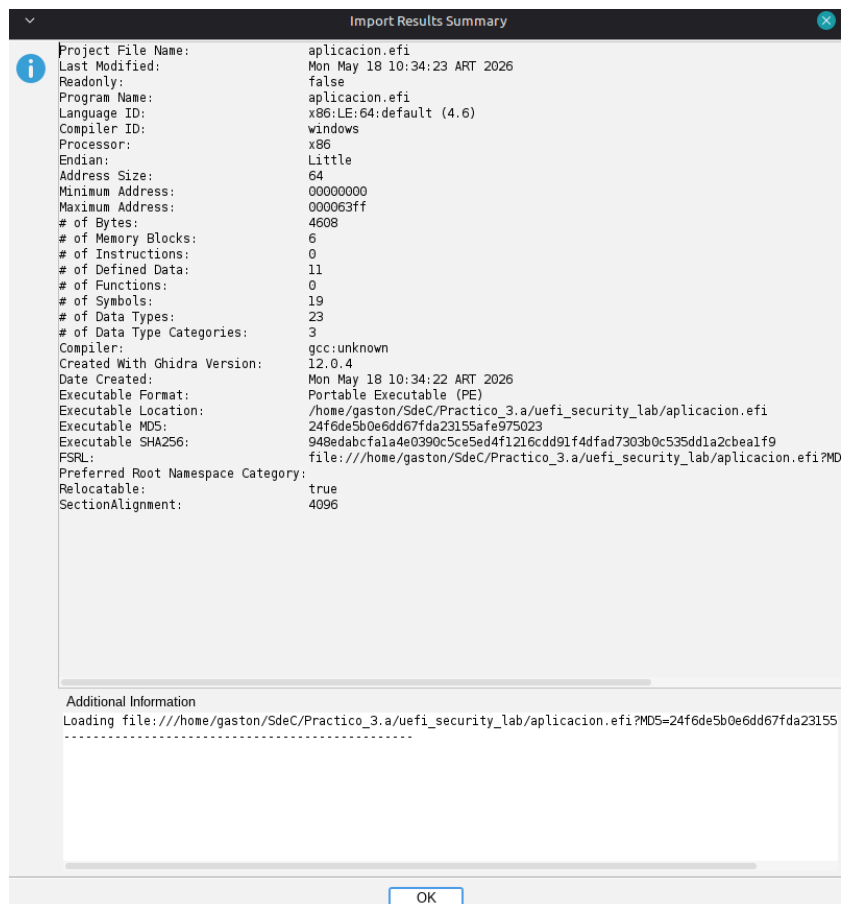


Figura 14: Información sobre el proyecto en GHIDRA.

Después de importar el archivo .efi se realiza el análisis:

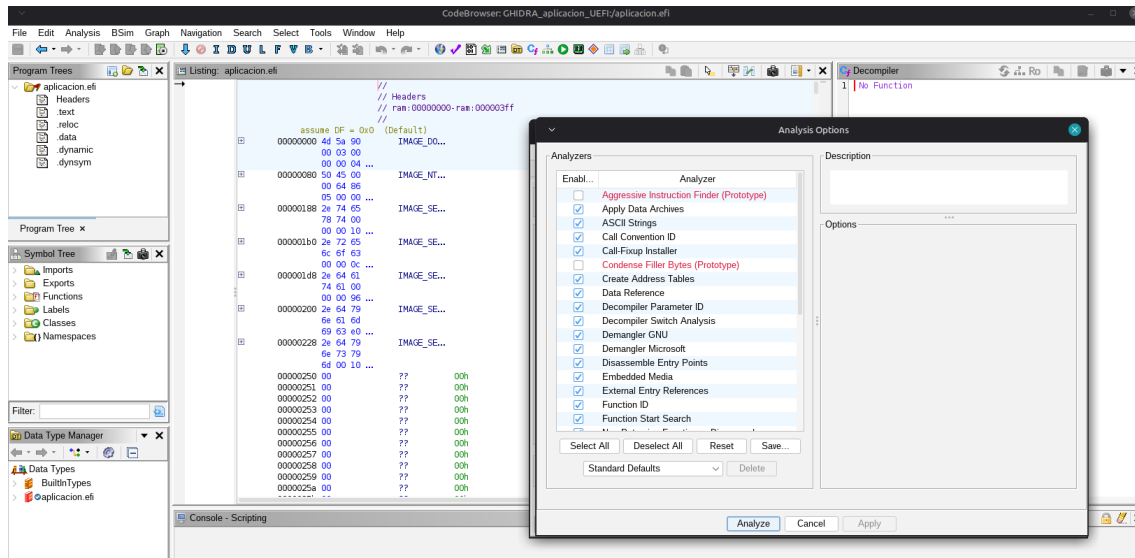


Figura 15: Análisis del archivo .efi en GHIDRA.

Se localiza la función efi.main:

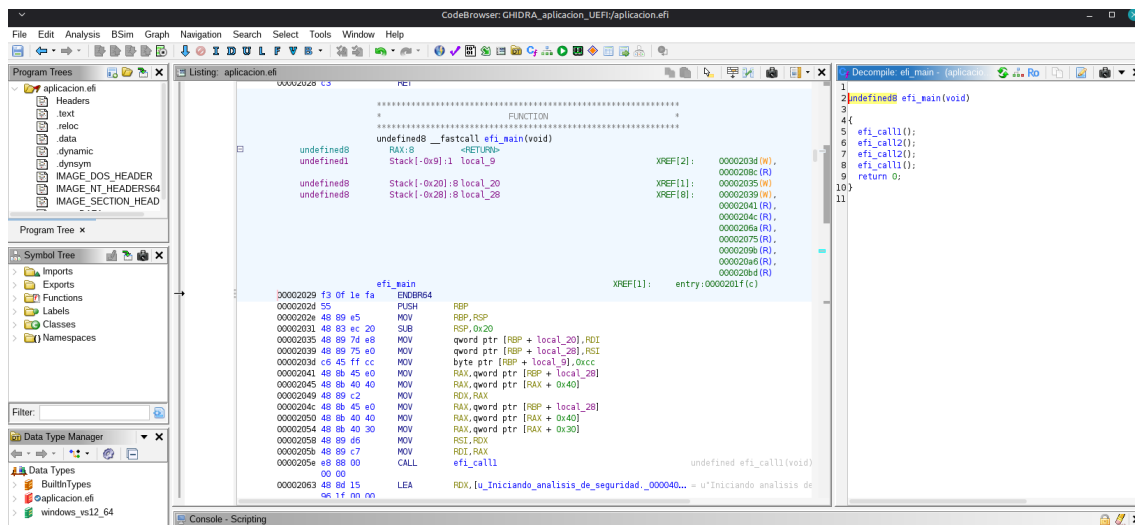


Figura 16: Función efi.main en GHIDRA.

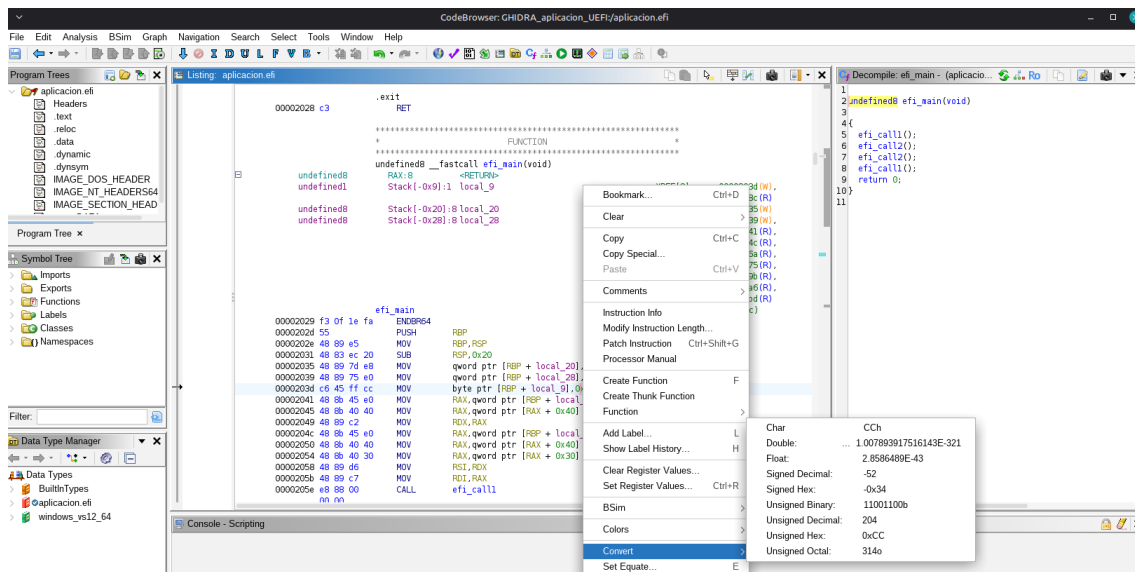


Figura 17: Relación entre 0xCC y -52.

Pregunta de Razonamiento 5: En el pseudocódigo de Ghidra, la condición 0xCC suele aparecer como -52. ¿A qué se debe este fenómeno y por qué importa en ciberseguridad?

Respuesta 5: Si se interpreta el número 0xCC (11001100 en binario) como número sin signo, entonces representa el 204, pero si se lo representa como número con signo representa el -52, tal como se ve en la Fig. 17.

Esto es importante en materia de ciberseguridad porque existen algunas reglas de detección automatizadas (como por ejemplo YARA o Weggli) que puede que intenter chequear el número incorrecto. Si un desarrollador usa un tipo char (con signo) para leer datos de red o archivos binarios (donde vienen bytes como 0xCC), y luego lo compara o lo copia en una variable más grande (como un int de 32 bits), el valor -52 se extiende conservando su signo (quedando como 0xFFFFFCC en lugar de 0x000000CC). Los atacantes utilizan estas conversiones implícitas de C/C++ para saltarse validaciones de longitud y provocar Desbordamientos de Búfer (Buffer Overflows).

2.4. Ejecutar aplicacion.efi en QEMU

Con el archivo aplicacion.efi ya compilado en la carpeta /uefi_security_lab/, se ejecutan los siguientes comandos:

```
# Primero se inició QEMU con la carpeta como disco virtual
qemu-system-x86_64 -m 512 \
  -bios /usr/share/ovmf/OVMF.fd \
  -net none \
  -drive file=fat:rw:$HOME/uefi_security_lab,format=raw
```

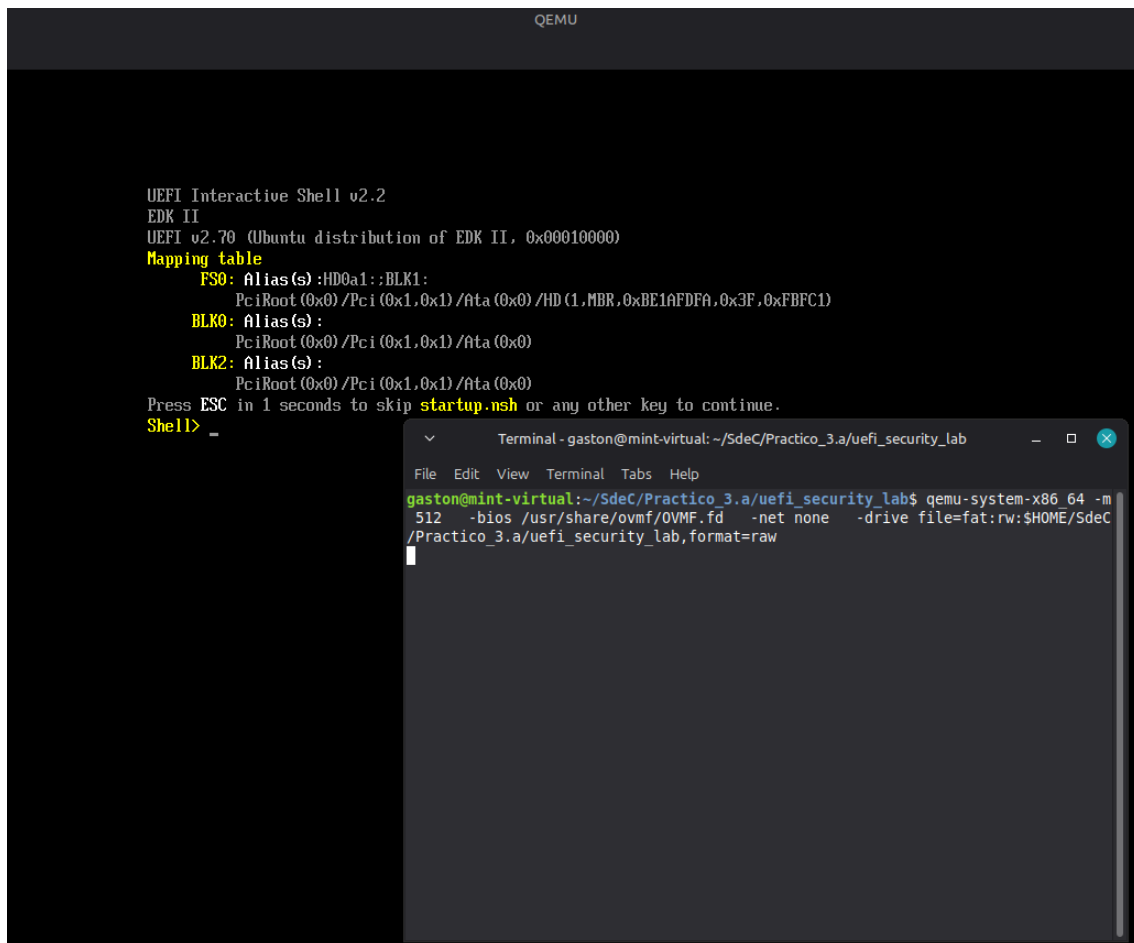


Figura 18: Ejecución de aplicacion.efi.

Una vez dentro de la UEFI Shell:

1. Verificar que el disco esté montado

Shell> map

```

Shell> map
Mapping table
FS0: Alias(s) :HD0a1::BLK1:
      PciRoot(0x0)/Pci(0x1,0x1)/Ata(0x0)/HD(1,MBR,0xBE1AFDFA,0x3F,0xFBFC1)
BLK0: Alias(s) :
      PciRoot(0x0)/Pci(0x1,0x1)/Ata(0x0)
BLK2: Alias(s) :
      PciRoot(0x0)/Pci(0x1,0x1)/Ata(0x0)
Shell> _
    
```

Figura 19: Comando map.

2. Entrar al filesystem

```
Shell> FS0:
```

3. Listar archivos (se debería ver aplicacion.efi)

```
FS0:\> ls
```

```
Shell> FS0:
FS0:\> ls
Directory of: FS0:\
05/18/2026  04:17                631  aplicacion.c
05/18/2026  04:18            5,158  aplicacion.efi
05/18/2026  04:17            1,928  aplicacion.o
05/18/2026  04:18           35,848  aplicacion.so
05/18/2026  10:25                0  GHIDRA_aplicacion_UEFI.gpr
05/18/2026  10:26 <DIR>         8,192  GHIDRA_aplicacion_UEFI.rep
05/18/2026  14:04           10,243  NvUars
               6 File(s)      53,808 bytes
               1 Dir(s)
```

Figura 20: Contenido de FS0.

4. Ejecutar la aplicación

```
FS0:0> aplicacion.efi
```

```
QEMU

Iniciando analisis de seguridad...
Breakpoint estatico alcanzado.
FS0:\> _
```

Figura 21: Ejecución de la aplicacion.efi.

3. Ejecución en Hardware Físico (Bare Metal)

Objetivo: Trasladar el binario compilado a una computadora real (ASUS X515EA) sorteando las restricciones del Secure Boot.

3.1. Preparación del Medio de Arranque (USB)

Primero es necesario identificar el pendrive con lsblk, y luego se ejecutan los siguientes comandos:

1. Formatear en FAT32 (requerimiento de la especificación UEFI)

```
sudo mkfs.vfat -F 32 /dev/sdb1
```

2. Montar el pendrive (ejemplo en /mnt)

```
sudo mount /dev/sdb1 /mnt
```

3. Crear la estructura estandarizada de directorios

```
sudo mkdir -p /mnt/EFI/BOOT
```

```
Terminal - gaston@mint-virtual: ~/SdeC/Practico_3.a/uefi_security_lab
File Edit View Terminal Tabs Help
gaston@mint-virtual:~/SdeC/Practico_3.a/uefi_security_lab$ lsblk -p
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
/dev/sda     8:0    0   25G  0 disk
├─/dev/sda1   8:1    0    1M  0 part
├─/dev/sda2   8:2    0   513M  0 part /boot/efi
├─/dev/sda3   8:3    0   24.5G  0 part /
└─/dev/sdb    8:16   1   953.3M  0 disk
    ├─/dev/sdb1 8:17   1   953.2M  0 part
    └─/dev/sr0 11:0    1   1024M  0 rom
gaston@mint-virtual:~/SdeC/Practico_3.a/uefi_security_lab$ sudo mkfs.vfat -F 32 /dev/sdb1
[sudo] password for gaston:
mkfs.fat 4.2 (2021-01-31)
gaston@mint-virtual:~/SdeC/Practico_3.a/uefi_security_lab$ sudo mount /dev/sdb1 /mnt
[sudo] password for gaston:
gaston@mint-virtual:~/SdeC/Practico_3.a/uefi_security_lab$ sudo mkdir -p /mnt/EFI/BOOT
gaston@mint-virtual:~/SdeC/Practico_3.a/uefi_security_lab$
```

Figura 22: Creación de estructura estandarizada de directorios.

4. Descargar la UEFI Shell oficial de TianoCore

```
sudo wget https://github.com/tianocore/edk2/raw/UDK2018/ShellBinPkg/UefiShell/X64/Shell.efi
-O /mnt/EFI/BOOT/BOOTX64.EFI
```

```
Terminal - gaston@mint-virtual: ~/SdeC/Practico_3.a/uefi_security_lab
File Edit View Terminal Tabs Help
gaston@mint-virtual:~/SdeC/Practico_3.a/uefi_security_lab$ sudo wget https://github.com/tiano
core/edk2/raw/UDK2018/ShellBinPkg/UefiShell/X64/Shell.efi -O /mnt/EFI/BOOT/BOOTX64.EFI
--2026-05-18 11:45:11-- https://github.com/tianocore/edk2/raw/UDK2018/ShellBinPkg/UefiShell/
X64/Shell.efi
Resolving github.com (github.com)... 4.228.31.150
Connecting to github.com (github.com)|4.228.31.150|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/tianocore/edk2/UDK2018/ShellBinPkg/UefiShell/X64/
Shell.efi [following]
--2026-05-18 11:45:11-- https://raw.githubusercontent.com/tianocore/edk2/UDK2018/ShellBinPkg
/UefiShell/X64/Shell.efi
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.1
10.133, 185.199.111.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... c
onected.
HTTP request sent, awaiting response... 200 OK
Length: 951744 (929K) [application/octet-stream]
Saving to: '/mnt/EFI/BOOT/BOOTX64.EFI'

/mnt/EFI/BOOT/BOOTX64.E 100%[=====] 929,44K --.-KB/s in 0,1s

2026-05-18 11:45:12 (6,55 MB/s) - '/mnt/EFI/BOOT/BOOTX64.EFI' saved [951744/951744]
gaston@mint-virtual:~/SdeC/Practico_3.a/uefi_security_lab$
```

Figura 23: UEFI Shell oficial de TianoCore.

5. Copiar aplicacion.efi a la raíz del pendrive

```
sudo cp ~/uefi_security_lab/aplicacion.efi /mnt/
sudo sync
sudo umount /mnt
```

```
gaston@mint-virtual:~/SdeC/Practico_3.a/uefi_security_lab$ sudo cp ~/SdeC/Practico_3.a/uefi_secur
ity_lab/aplicacion.efi /mnt/
gaston@mint-virtual:~/SdeC/Practico_3.a/uefi_security_lab$ sudo sync
gaston@mint-virtual:~/SdeC/Practico_3.a/uefi_security_lab$ sudo umount /mnt
gaston@mint-virtual:~/SdeC/Practico_3.a/uefi_security_lab$
```

Figura 24: Copia de aplicacion.efi al pendrive.

3.2. Configuración del BIOS/UEFI (ASUS X515EA)

1. Conectar el USB y encender la laptop presionando F2.
2. Ir a Advanced Mode (F7).

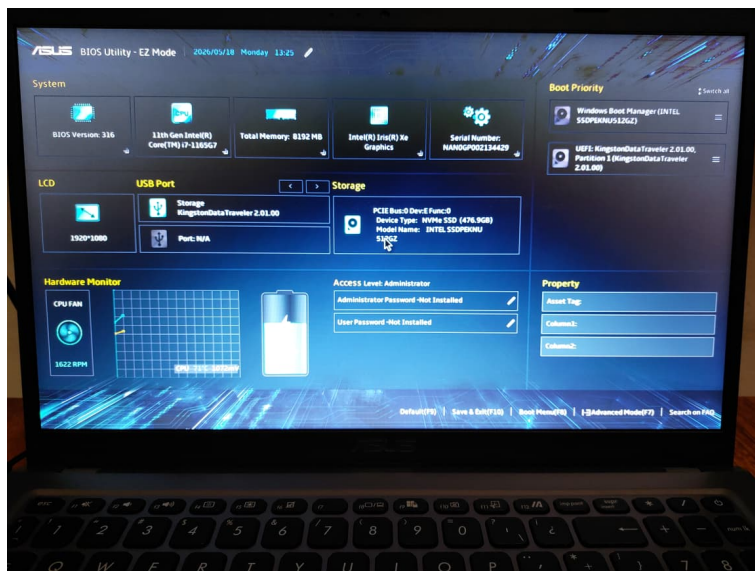


Figura 25: UEFI.

3. Ir a Security -> Secure Boot and configurarlo en Disabled. (Explicación: El binario de la Shell y la aplicacion.efi no están firmados criptográficamente por Microsoft/ASUS. Si se deja Secure Boot activo, el firmware rechazará la ejecución devolviendo un error de violación de seguridad).

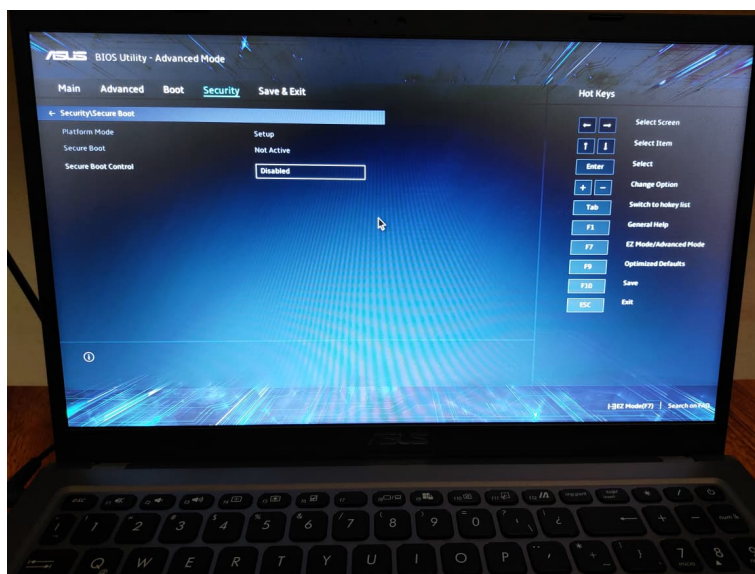


Figura 26: Deshabilitación del Boot Control.

4. Guardar los cambios con F10.

3.3. Ejecución en Bare Metal

1. Reiniciar presionando esc para abrir el Boot Menu.
2. Seleccionar el pendrive USB. El sistema iniciará la Shell (BOOTX64.EFI).

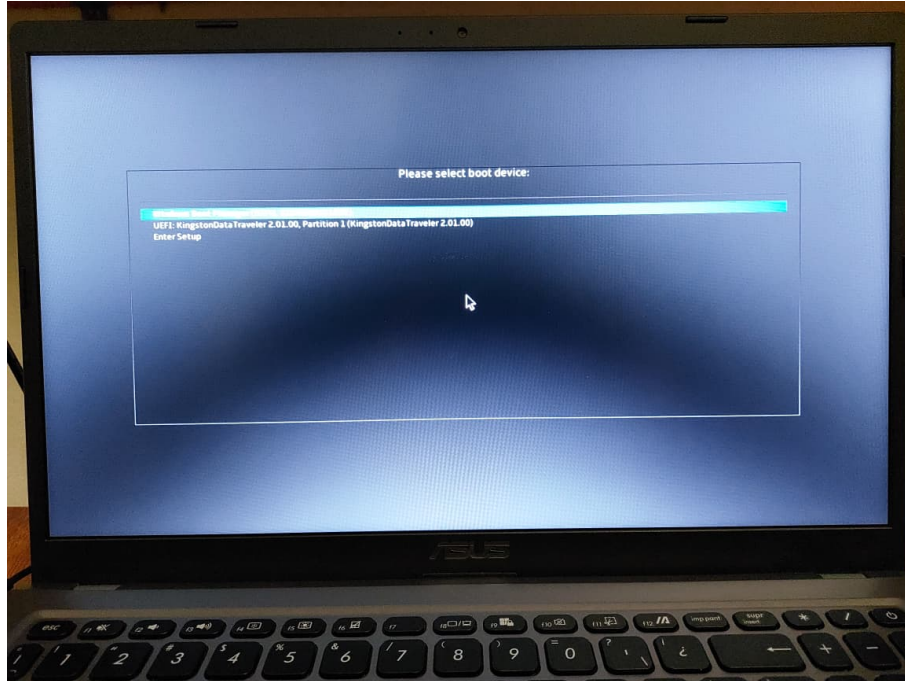


Figura 27: Boot Menu.

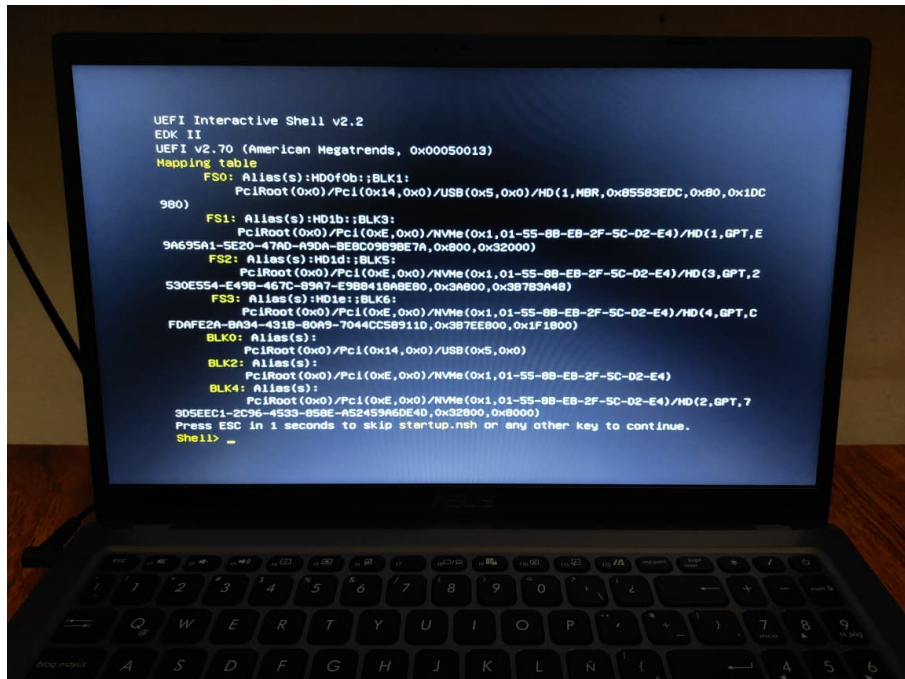


Figura 28: UEFI Shell.

3. En el prompt de la shell, cambiar al sistema de archivos del pendrive y ejecutar la aplicación:

```
Shell> FS0:
FS0:\> ls
FS0:\> aplicacion.efi
```

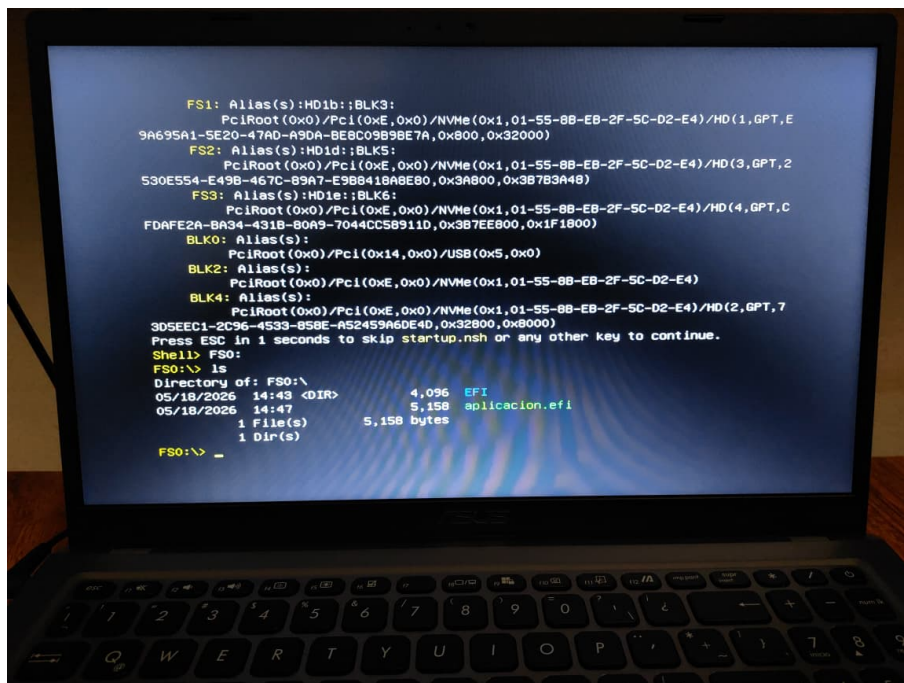


Figura 29: Comando ls.

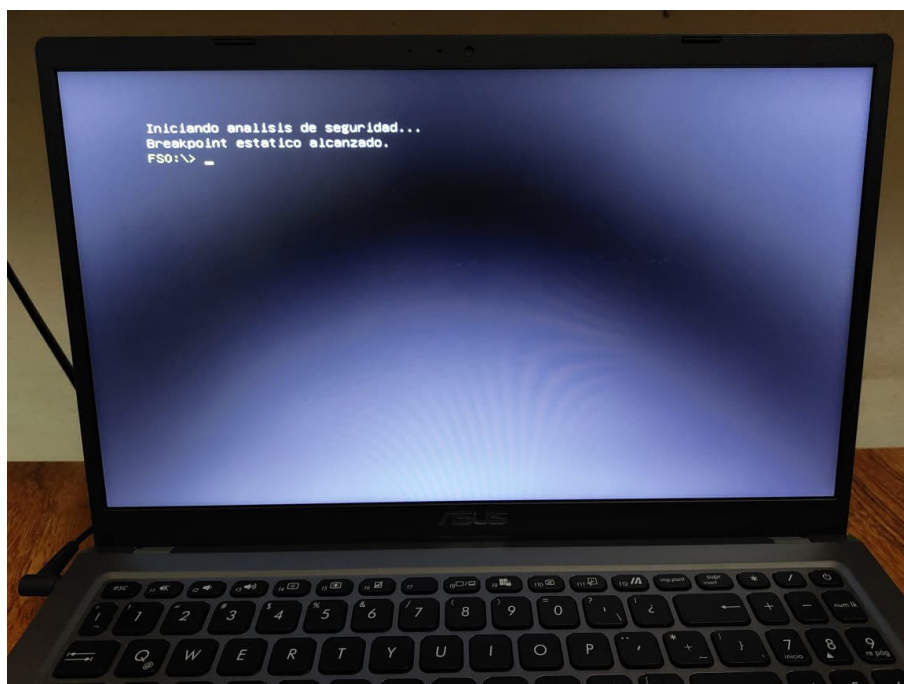


Figura 30: Ejecución de aplicacion.efi.

4. Depuración con GDB

4.1. Depuración con GDB más QEMU

Iniciar QEMU con el servidor GDB habilitado:

```
qemu-system-x86_64 -m 512 \
-bios /usr/share/ovmf/OVMF.fd \
-net none \
-drive file=fat:rw:$HOME/uefi_security_lab,format=raw \
-s \
-S
```

-s es shorthand de -gdb tcp::1234

-S hace que QEMU no arranque hasta que GDB le diga continue

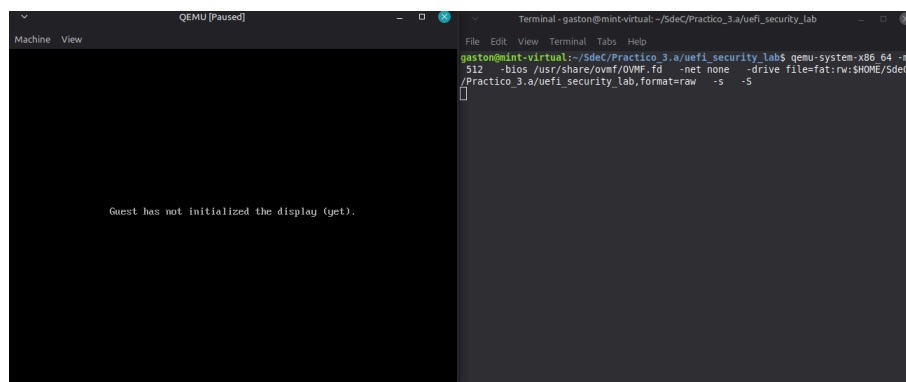


Figura 31: Inicio de QEMU a la espera de GDB.

Conectar GDB

En otra terminal:

```
gdb
(gdb) target remote localhost:1234
# Dejar que el firmware inicialice hasta llegar a la UEFI Shell
(gdb) continue
```

El problema: encontrar la dirección base del archivo .efi.

UEFI carga las imágenes en direcciones dinámicas (no fijas), entonces es necesario saber dónde en memoria se cargó la aplicación .efi.

La alternativa más práctica es agregar un bucle al inicio de la app para que se quede esperando, y mientras tanto leer la dirección desde GDB. Para ello se copia el archivo aplicación.c (obteniendo aplicación2.c) y se agrega el siguiente bucle al inicio:

```
cEFI_STATUS EFIAPI efi_main(EFI_HANDLE ImageHandle, EFI_SYSTEM_TABLE *SystemTable) {
    volatile int waiting = 1;
    while (waiting) {} // GDB conecta acá y cambia waiting a 0
    // ... resto del código
}
```

```

QEMU

UEFI Interactive Shell v2.2
EDK II
UEFI v2.70 (Ubuntu distribution of EDK II, 0x00010000)
Mapping table
  FS0: Alias(s) :HD0a1::BLK1:
      PciRoot(0x0)/Pci(0x1,0x1)/Ata(0x0)/HD(1,MBR,0xBE1AFDFA,0x3F,0xFBFC1)
  BLK0: Alias(s) :
      PciRoot(0x0)/Pci(0x1,0x1)/Ata(0x0)
  BLK2: Alias(s) :
      PciRoot(0x0)/Pci(0x1,0x1)/Ata(0x0)
Press ESC in 1 seconds to skip startup.nsh or any other key to continue.
Shell> FS0:
FS0:\> ls
Directory of: FS0:\
05/18/2026  16:13          541  .gdb_history
05/18/2026  14:14          631  aplicacion.c
05/18/2026  14:14       5,158  aplicacion.efi
05/18/2026  14:14       1,928  aplicacion.o
05/18/2026  13:57      35,848  aplicacion.so
05/18/2026  16:13         737  aplicacion2.c
05/18/2026  15:59       5,158  aplicacion2.efi
05/18/2026  15:59       1,944  aplicacion2.o
05/18/2026  14:27      35,848  aplicacion2.so
05/18/2026  16:24     192,485  Arquitectura y Fundamentos del Firmware UEFI .pdf
05/18/2026  14:13           0  debug.log
05/18/2026  10:25           0  GHIDRA_aplicacion_UEFI.gpr
05/18/2026  10:26 <DIR>       8,192  GHIDRA_aplicacion_UEFI.rep
05/18/2026  23:54       10,243  NoVars
               13 File(s)    290,521 bytes
                1 Dir(s)
FS0:\> _
    
```

Figura 32: Contenido de FS0 (ahora también está aplicacion2.efi).

Luego se ejecuta la aplicacion2.efi.

En GDB:

Pausar la ejecución

ctrl + C

Ver instrucciones

x/20i \$rip

Breakpoint cuando ya se cargó el 0xCC

break *0x1e072050

Se setea en cero la variable "waiting" para salir del bucle

set {int}(\$rbp-x04) = 0


```

Terminal - gaston@mint-virtual: ~/SdeC/Practico_3.a/uefi_security_lab
File Edit View Terminal Tabs Help
>>> x/20i $rip
=> 0x1e072045 <efi_main+28>: mov     -0x4(%rbp),%eax
0x1e072048 <efi_main+31>: test    %eax,%eax
0x1e07204a <efi_main+33>: jne     0x1e072045 <efi_main+28>
0x1e07204c <efi_main+35>: movb    $0xcc,-0x5(%rbp)
0x1e072050 <efi_main+39>: mov     -0x20(%rbp),%rax
0x1e072054 <efi_main+43>: mov     0x40(%rax),%rax
0x1e072058 <efi_main+47>: mov     %rax,%rdx
0x1e07205b <efi_main+50>: mov     -0x20(%rbp),%rax
0x1e07205f <efi_main+54>: mov     0x40(%rax),%rax
0x1e072063 <efi_main+58>: mov     0x30(%rax),%rax
0x1e072067 <efi_main+62>: mov     %rdx,%rsi
0x1e07206a <efi_main+65>: mov     %rax,%rdi
0x1e07206d <efi_main+68>: call    0x1e0720fa <efi_call1>
0x1e072072 <efi_main+73>: lea     0x1f87(%rip),%rdx      # 0x1e074000
0x1e072079 <efi_main+80>: mov     -0x20(%rbp),%rax
0x1e07207d <efi_main+84>: mov     0x40(%rax),%rax
0x1e072081 <efi_main+88>: mov     %rax,%rcx
0x1e072084 <efi_main+91>: mov     -0x20(%rbp),%rax
0x1e072088 <efi_main+95>: mov     0x40(%rax),%rax
0x1e07208c <efi_main+99>: mov     0x8(%rax),%rax
>>> break *0x1e072050
Breakpoint 1 at 0x1e072050
>>> set {int}($rbp-0x4) = 0
>>>
  
```

Figura 33: info.

Ahora se puede ejecutar el comando continue para continuar la ejecución del programa, que va a salir del bucle y se va a frenar en el breakpoint:

continue

Luego se comprueba que se cargó el valor 0xCC:

```

Terminal - gaston@mint-virtual: ~/SdeC/Practico_3.a/uefi_security_lab
File Edit View Terminal Tabs Help
[3] from 0x000000001fe9fab6
[4] from 0x000000001fe964f0
[5] from 0x0000000000000000
Threads
[1] id 1 from 0x000000001e072050 in efi_main
Variables
>>> x/lbx $rbp-0x5
0x1fe9646b: 0xcc
>>> info registers rax rbp rsp
rax      0x0      0
rbp      0x1fe96470 0x1fe96470
rsp      0x1fe96450 0x1fe96450
>>> x/10i $rip
=> 0x1e072050 <efi_main+39>: mov     -0x20(%rbp),%rax
0x1e072054 <efi_main+43>: mov     0x40(%rax),%rax
0x1e072058 <efi_main+47>: mov     %rax,%rdx
0x1e07205b <efi_main+50>: mov     -0x20(%rbp),%rax
0x1e07205f <efi_main+54>: mov     0x40(%rax),%rax
0x1e072063 <efi_main+58>: mov     0x30(%rax),%rax
0x1e072067 <efi_main+62>: mov     %rdx,%rsi
0x1e07206a <efi_main+65>: mov     %rax,%rdi
0x1e07206d <efi_main+68>: call    0x1e0720fa <efi_call1>
0x1e072072 <efi_main+73>: lea     0x1f87(%rip),%rdx      # 0x1e074000
>>>
  
```

Figura 34: Se verifica que se cargó 0xCC y se ven los registros.

4.2. Conectar Ghidra a GDB (Análisis Híbrido)

Ghidra tiene un debugger integrado desde la versión 10+ que puede conectarse a QEMU vía GDB remote. Configurar en Ghidra

1. Se abre el proyecto con aplicacion2.efi ya importado y analizado
2. Menú: Debugger → Configure and Launch aplicacion2.efi
3. En el diálogo que aparece, se elige: “gdb remote”
4. Configuración:

Host: localhost

Port: 1234

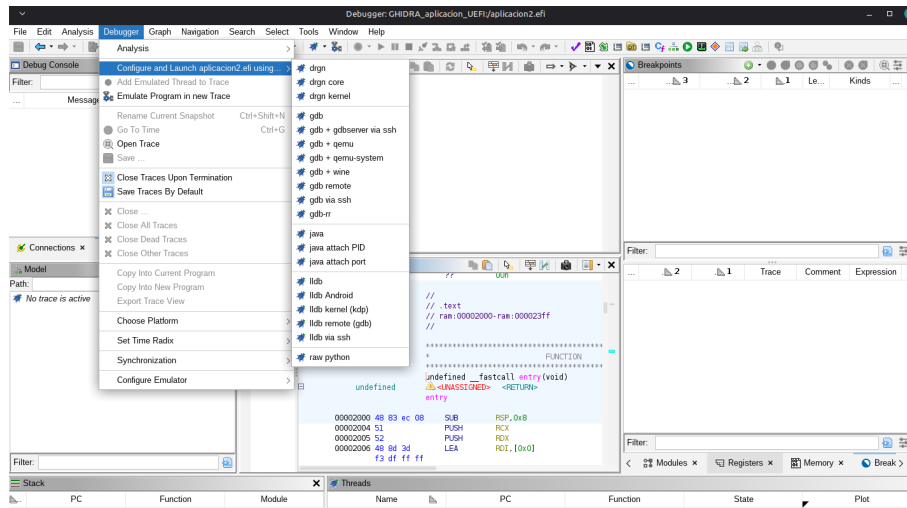


Figura 35: Configuración del Debugger.

1. Se inicia QEMU con `-s -S` -> queda pausado
2. Se abre Ghidra con el .efi importado
3. Se conecta Ghidra al GDB de QEMU
4. En la UEFI Shell se ejecuta aplicacion2.efi (QEMU la carga pero Ghidra la para en el entry point)
5. En Ghidra, se navega a efi_main en el listing y se pone un breakpoint en la línea del 0xCC

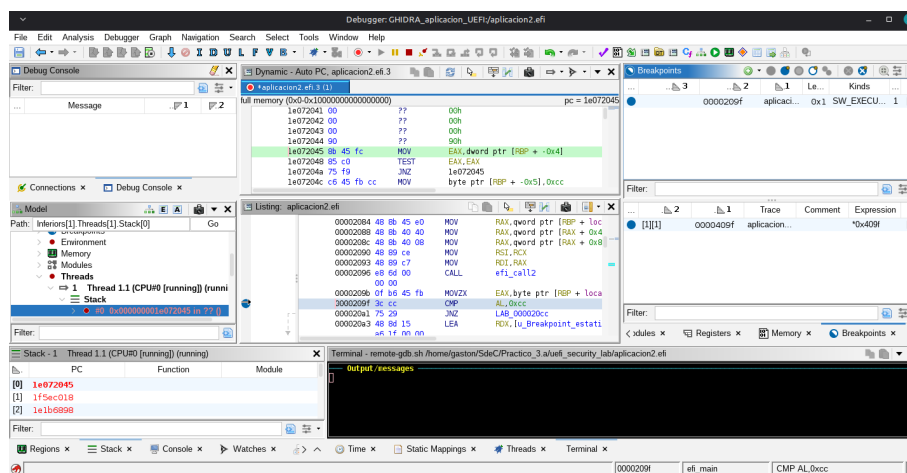


Figura 36: Configuración del breakpoint.

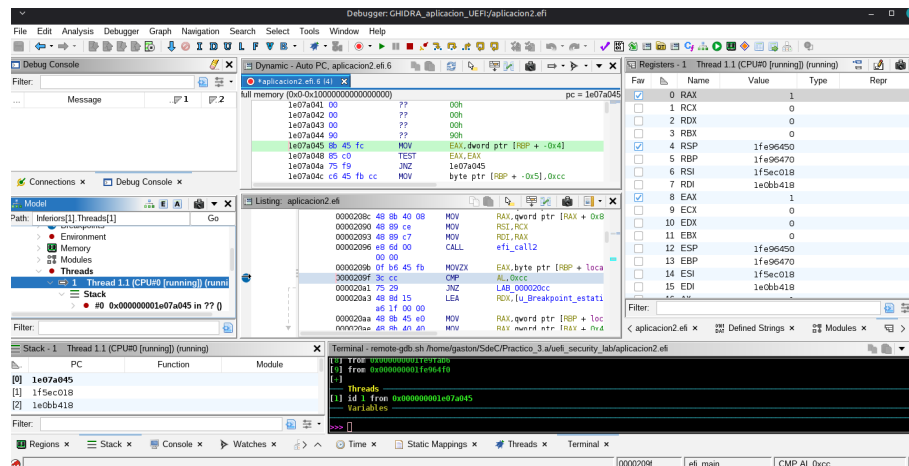


Figura 37: Se verifica que la variable waiting está en 1.

Se observa que en este punto la variable “waiting” está en 1 (cargada en el registro RAX), por lo que aún se está dentro del bucle while. En este caso no se pudo cambiar el valor de la variable en el entorno de GHIDRA.

5. Resumen

Pregunta de Razonamiento 1: Al ejecutar el comando map y dh, vemos protocolos e identificadores en lugar de puertos de hardware fijos. ¿Cuál es la ventaja de seguridad de este modelo frente al antiguo BIOS?

Respuesta 1: El antiguo BIOS funcionaba en modo real de 16 bits, de forma que cualquier programa que se ejecutara durante el arranque tenía acceso a toda la memoria y los puertos de hardware.

Pregunta de Razonamiento 2: Observando las variables Boot#### y BootOrder, ¿cómo determina el Boot Manager la secuencia de arranque?

Respuesta 2: Lo primero que hace el Boot Manager es leer la variable BootOrder, la cual contiene una lista ordenada de números hexadecimales de 4 dígitos. Luego, el Boot Manager se encarga de buscar la variable Boot#### (uno de los números de la lista en BootOrder), la cual contiene la ruta hacia el archivo ejecutable del cargador del arranque. Si el intento de arranque falla, se elige otra variable dentro de la lista de BootOrder y se repite el proceso.

Pregunta de Razonamiento 3: En el mapa de memoria (memmap), existen regiones marcadas como RuntimeServicesCode. ¿Por qué estas áreas son un objetivo principal para los desarrolladores de malware (Bootkits)?

Respuesta 3: Mientras que la memoria de Boot Services se borra cuando se termina de encender el equipo, la memoria de Runtime Services contiene funciones críticas que el SO necesita consultar durante el funcionamiento. Entonces, si se logra inyectar un código malicioso en esta región de memoria, el virus sobrevive el proceso de arranque, y también tiene los privilegios más altos posibles.

Pregunta de Razonamiento 4: ¿Por qué utilizamos SystemTable->ConOut->OutputString en lugar de la función printf de C?

Respuesta 4: La función printf pertenece a la biblioteca de C <stdio.h>, y se encarga de comunicarse con el SO usando system calls para lograr que se dibujen letras en la pantalla. En cambio, cuando se ejecuta el código UEFI, el SO está apagado, por lo que no hay bibliotecas cargadas en memoria, ni hay un kernel al cual pedirle que imprima cosas. Es por ello que es necesario utilizar la SystemTable, que contiene funciones primitivas como OutputString.

Respuesta 5: Si se interpreta el número 0xCC (11001100 en binario) como número sin signo, entonces representa el 204, pero si se lo representa como número con signo representa el -52, tal como se ve en la Fig. 17.

Esto es importante en materia de ciberseguridad porque existen algunas reglas de detección automatizadas (como por ejemplo YARA o Weggli) que puede que intenter chequear el número incorrecto. Si un desarrollador usa un tipo char (con signo) para leer datos de red o archivos binarios (donde vienen bytes como 0xCC), y luego lo compara o lo copia en una variable más grande (como un int de 32 bits), el valor -52 se extiende conservando su signo (quedando como 0xFFFFFCC en lugar de 0x000000CC). Los atacantes utilizan estas conversiones implícitas de C/C++ para saltarse validaciones de longitud y provocar Desbordamientos de Búfer (Buffer Overflows).

6. Link al Repositorio

<https://github.com/GastonYomaha/SistemasDeComputacion/tree/main/TP3.a>